

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

ДЕНЬ 12

10 вариант

**Отладка программы с использованием специализированных средств
отладки**

Выполнила: Хлебова Дарья,

Студентка 24-ИС.

Руководитель: Торопов Е.В.

р.п. Красные Баки

2026 г.

```

import csv
import os
import tracemalloc
from typing import List, Dict, Any

# Глобальный кеш для хранения содержимого файлов (утечка памяти)
FILE_CACHE = {}

def read_csv_file(filename: str) -> List[Dict[str, Any]]:
    """
    Читает CSV-файл и возвращает список словарей.
    ОШИБКА 1: Нет проверки существования файла
    ОШИБКА 2: Не закрывается файловый дескриптор при ошибке
    """
    breakpoint()
    # Ошибка: если файл не существует - будет FileNotFoundError
    file = open(filename, 'r', encoding='utf-8')

    # Ошибка: чтение всего файла в память без учета размера
    content = file.read()

    # Ошибка: накопление в глобальном кеше без ограничения
    FILE_CACHE[filename] = content

    # Ошибка: парсинг CSV без обработки ошибок формата
    reader = csv.DictReader(content.splitlines())
    result = []
    for row in reader:
        # Ошибка: неверное преобразование типов (если поле пустое)

```

```
        row['value'] = float(row['value']) # ValueError если 'value' не число или
пусто
    result.append(row)
```

```
# Ошибка: файл не закрывается, если произошло исключение выше
file.close()
return result
```

```
def process_data(data: List[Dict[str, Any]], multiplier: float) -> List[Dict[str, Any]]:
    """
```

Обрабатывает данные: умножает значение на множитель.

ОШИБКА 3: Логическая ошибка - неправильное преобразование типов

```
    """
```

```
    for item in data:
```

```
        # Ошибка: если multiplier передан как строка, будет TypeError
```

```
        # или если значение не число
```

```
        item['value'] = item['value'] * multiplier
```

```
# Ошибка: накопление в кеше даже после обработки
```

```
FILE_CACHE['processed'] = data
```

```
return data
```

```
def save_results(filename: str, data: List[Dict[str, Any]]) -> None:
```

```
    """
```

Сохраняет обработанные данные в файл.

ОШИБКА 4: Контекстный менеджер с ошибкой - неправильное использование

```
    """
```

```
# Ошибка: использование контекстного менеджера с ошибкой в режиме
```

```
# Если файл не может быть создан (нет прав) - исключение не
обрабатывается
```

```
with open(filename, 'w', encoding='utf-8') as file:
```

```
    writer = csv.DictWriter(file, fieldnames=['value'])
```

```
    writer.writeheader()
```

```
    for item in data:
```

```
        # Ошибка: запись неверного формата (может вызвать исключение)
```

```
        writer.writerow({'value': str(item['value'])}) # Правильно: item['value']
```

```
def main():
```

```
    """Основная функция с демонстрацией ошибок."""
```

```
    tracemalloc.start()
```

```
    # Тестовые данные - файл может не существовать
```

```
    input_file = 'data.csv'
```

```
    output_file = 'result.csv'
```

```
    print("=== Начало обработки ===")
```

```
    try:
```

```
        # Шаг 1: Чтение файла
```

```
        print(f"Чтение файла: {input_file}")
```

```
        data = read_csv_file(input_file)
```

```
        print(f"Прочитано {len(data)} записей")
```

```
        # Шаг 2: Обработка данных
```

```
        print("Обработка данных...")
```

```
        processed = process_data(data, 2.5)
```

```

# Шаг 3: Сохранение результата
print(f'Сохранение в: {output_file}')
save_results(output_file, processed)

print("Обработка завершена успешно!")

except FileNotFoundError as e:
    print(f'ОШИБКА: Файл не найден - {e}')
    # Ошибка: файловый дескриптор может остаться открытым
except ValueError as e:
    print(f'ОШИБКА: Неверный формат данных - {e}')
except Exception as e:
    print(f'НЕИЗВЕСТНАЯ ОШИБКА: {e}')
    import traceback
    traceback.print_exc()
finally:
    # Вывод статистики памяти
    snapshot = tracemalloc.take_snapshot()
    print("\n=== Топ-5 строк по потреблению памяти ===")
    for stat in snapshot.statistics('lineno')[:5]:
        print(stat)

    print(f"\nРазмер кеша: {len(FILE_CACHE)} записей")

if __name__ == "__main__":
    main()

```

```

=====
=== Начало обработки ===
Чтение файла: data.csv
ОШИБКА: файл не найден - [Errno 2] No such file or directory: 'data.csv'

=== Топ-5 строк по потреблению памяти ===
C:\Users\Student235\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:388: size=15.7 KiB, count=177, average=91 B
C:\Users\Student235\AppData\Local\Programs\Python\Python313\Lib\threading.py:297: size=760 B, count=2, average=380 B
Y:\24 ИСП 2025\УП02\Группа 2\Хлебцова Д\12день\variant_10.py:55: size=280 B, count=1, average=280 B
C:\Users\Student235\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:230: size=192 B, count=1, average=192 B
C:\Users\Student235\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:465: size=160 B, count=1, average=160 B

Размер кеша: 0 записей

```

Тип ошибки: FileNotFoundError

```
import csv
```

```
import os
```

```
import tracemalloc
```

```
import functools
```

```
from typing import List, Dict, Any
```

```
# Используем LRU-кеш вместо глобального словаря
```

```
@functools.lru_cache(maxsize=128)
```

```
def read_file_content(filename: str) -> str:
```

```
    """Читает содержимое файла с кешированием."""
```

```
    with open(filename, 'r', encoding='utf-8') as f:
```

```
        return f.read()
```

```
def read_csv_file(filename: str) -> List[Dict[str, Any]]:
```

```
    """Читает CSV-файл и возвращает список словарей."""
```

```
    # Проверка существования файла
```

```
    if not os.path.exists(filename):
```

```
        print(f"Предупреждение: Файл {filename} не найден. Создаю тестовый файл.")
```

```
        with open(filename, 'w', encoding='utf-8') as f:
```

```
            f.write("value\n100\n200\n300\n")
```

```

# Чтение с кешированием
try:
    content = read_file_content(filename)
except FileNotFoundError:
    # Повторная попытка с созданием
    with open(filename, 'w', encoding='utf-8') as f:
        f.write("value\n100\n200\n300\n")
    content = read_file_content(filename)

# Парсинг CSV с обработкой ошибок
reader = csv.DictReader(content.splitlines())
result = []
for idx, row in enumerate(reader):
    try:
        if 'value' not in row or not row['value'].strip():
            print(f"Предупреждение: строка {idx} содержит пустое значение")
            continue
        row['value'] = float(row['value'])
    except ValueError as e:
        print(f"Предупреждение: строка {idx} - неверный формат: {row.get('value', '')}")
        continue
    result.append(row)

return result

def process_data(data: List[Dict[str, Any]], multiplier: float) -> List[Dict[str, Any]]:
    """Обрабатывает данные: умножает значение на множитель."""
    # Проверка типа multiplier

```

```
if not isinstance(multiplier, (int, float)):
    try:
        multiplier = float(multiplier)
    except (ValueError, TypeError):
        print(f"Ошибка: multiplier должен быть числом, получен {type(multiplier)}")
        multiplier = 1.0
```

```
processed = []
for item in data:
    if 'value' not in item:
        print(f"Предупреждение: элемент без ключа 'value': {item}")
        continue
    if not isinstance(item['value'], (int, float)):
        try:
            item['value'] = float(item['value'])
        except (ValueError, TypeError):
            print(f"Предупреждение: неверное значение: {item['value']}")
            continue
    # Создаем копию, чтобы не изменять оригинал
    new_item = item.copy()
    new_item['value'] = new_item['value'] * multiplier
    processed.append(new_item)
```

```
return processed
```

```
def save_results(filename: str, data: List[Dict[str, Any]]) -> None:
    """Сохраняет обработанные данные в файл."""
    try:
```



```

with open(filename, 'w', encoding='utf-8') as file:
    writer = csv.DictWriter(file, fieldnames=['value'])
    writer.writeheader()
    for item in data:
        if 'value' in item:
            writer.writerow({'value': item['value']})
        else:
            print(f"Предупреждение: пропуск элемента без 'value': {item}")
    print(f"Файл {filename} успешно сохранен")
except (IOError, PermissionError) as e:
    print(f"Ошибка сохранения файла {filename}: {e}")
    # Создаем резервную копию
    with open('backup.json', 'w', encoding='utf-8') as f:
        import json
        json.dump(data, f, indent=2)
    print("Создана резервная копия в backup.json")
    raise

```

```

def main():
    """Основная функция с демонстрацией ошибок."""
    tracemalloc.start()

    input_file = 'data.csv'
    output_file = 'result.csv'

    print("=== Начало обработки ===")

    try:
        # Шаг 1: Чтение файла

```

```
print(f"Чтение файла: {input_file}")
data = read_csv_file(input_file)
print(f"Прочитано {len(data)} записей")
```

```
# Шаг 2: Обработка данных
print("Обработка данных...")
processed = process_data(data, 2.5)
print(f"Обработано {len(processed)} записей")
```

```
# Шаг 3: Сохранение результата
print(f"Сохранение в: {output_file}")
save_results(output_file, processed)
```

```
print("Обработка завершена успешно!")
```

```
except FileNotFoundError as e:
```

```
    print(f"ОШИБКА: Файл не найден - {e}")
```

```
except ValueError as e:
```

```
    print(f"ОШИБКА: Неверный формат данных - {e}")
```

```
except Exception as e:
```

```
    print(f"НЕИЗВЕСТНАЯ ОШИБКА: {e}")
```

```
    import traceback
```

```
    traceback.print_exc()
```

```
finally:
```

```
    # Вывод статистики памяти
```

```
    snapshot = tracemalloc.take_snapshot()
```

```
    print("\n=== Топ-5 строк по потреблению памяти ===")
```

```
    for stat in snapshot.statistics('lineno')[:5]:
```

```
        print(stat)
```

```
print(f"\nРазмер кеша: {read_file_content.cache_info()}")
```

```
if __name__ == "__main__":
```

```
    main()
```

```
=== Начало обработки ===
Чтение файла: data.csv
Прочитано 3 записей
Обработка данных...
Обработано 3 записей
Сохранение в: result.csv
файл result.csv успешно сохранен
Обработка завершена успешно!

=== Топ-5 строк по потреблению памяти ===
C:\Users\Student235\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:3
88: size=15.7 KiB, count=177, average=91 B
C:\Users\Student235\AppData\Local\Programs\Python\Python313\Lib\threading.py:297
: size=760 B, count=2, average=380 B
Y:/24 ИСиП 2025/УП02/Группа 2/Хлебова Д/12день/ispr.py:69: size=488 B, count=5,
average=98 B
C:\Users\Student235\AppData\Local\Programs\Python\Python313\Lib\csv.py:186: size
=432 B, count=5, average=86 B
Y:/24 ИСиП 2025/УП02/Группа 2/Хлебова Д/12день/ispr.py:78: size=299 B, count=3,
average=100 B

Размер кеша: CacheInfo(hits=0, misses=1, maxsize=128, currsize=1)
```

КОНТРОЛЬНЫЕ ВОПРОСЫ

A1. Что такое стек вызовов (call stack) в Python и как его увидеть?

Стек вызовов - это структура данных, хранящая последовательность вызовов функций в момент выполнения программы. Увидеть его можно через `traceback.print_exc()` или при возникновении исключения, когда Python автоматически выводит `traceback`.

A2. В чем разница между `except:` и `except Exception as e:`?
`except:` перехватывает все исключения, включая системные (`KeyboardInterrupt`, `SystemExit`), что может скрыть критические ошибки. `except Exception as e:` перехватывает только стандартные исключения, оставляя системные для обработки выше.

A3. Как запустить Python-скрипт под отладчиком pdb без изменения исходного кода?

Использовать команду: `python -m pdb script.py`

A4. Какие основные команды pdb вы знаете и для чего они нужны?

- `n (next)` - выполнить следующую строку
- `s (step)` - войти в функцию
- `c (continue)` - продолжить до следующего брейкпоинта
- `p var` - напечатать переменную
- `pp` - красивая печать
- `l` - показать код вокруг текущей строки
- `up/down` - перемещение по стеку вызовов
- `break` - установка брейкпоинта
- `condition` - условный брейкпоинт

A5. Как установить точку останова (breakpoint) в pdb на определенной строке или функции?

`break function_name` или `break filename:line_number`

A6. Что такое утечка памяти в Python и как она может возникнуть?
Утечка памяти - когда объекты не освобождаются сборщиком мусора, хотя больше не используются. Возникает при: глобальных кешах без ограничения, замыканиях, циклических ссылках, незакрытых файлах/соединениях.

A7. Какой модуль в Python позволяет отслеживать выделение памяти?
`tracemalloc` - встроенный модуль для трассировки выделений памяти.

A8. Как вывести красивый traceback ошибки, чтобы понять, где именно произошло исключение?
`import traceback; traceback.print_exc()` или использовать `logging.exception()`

A9. Что такое `RecursionError` и как его избежать?
`RecursionError` возникает при превышении максимальной глубины рекурсии

(по умолчанию 1000). Избегать можно: установкой `sys.setrecursionlimit()`, преобразованием рекурсии в итерацию, добавлением корректного базового случая.

A10. Для чего нужна команда `up` в `pdb`?
Для перемещения на один уровень вверх по стеку вызовов (к родительской функции), чтобы просмотреть контекст вызвавшей функции.

B1. В чем разница между `breakpoint()` и `import pdb; pdb.set_trace()`?
`breakpoint()` появился в Python 3.7 и является более удобным: он использует настройки из переменной окружения `PYTHONBREAKPOINT` и может быть отключен глобально. `pdb.set_trace()` жестко привязан к `pdb`.

B2. Как в `pdb` продолжить выполнение до того момента, когда некоторая переменная станет равной определенному значению, не используя `while` в коде?

Использовать условный брейкпоинт:
`break filename:line, condition var == value`
`condition 1 var == value`

B3. Почему использование `print()` для отладки считается непрофессиональным и к каким проблемам это может привести?

- Засоряет код и вывод программы
- Требуется удаление после отладки
- Не дает интерактивности (нельзя менять переменные)
- Не работает в продакшене без перезапуска
- Не имеет уровней логирования
- Может замедлять выполнение (особенно при частых вызовах)

B4. Как использовать `tracemalloc` для поиска утечек памяти в долгоживущем приложении (например, веб-сервер)?

1. Запустить `tracemalloc.start()` при старте приложения

2. Периодически снимать снимки: `snapshot1 = tracemalloc.take_snapshot()`
3. После каждого запроса/цикла снимать новый снимок и сравнивать: `snapshot2.compare_to(snapshot1, 'lineno')`
4. Отслеживать рост памяти по строкам кода
5. Настроить автоматический отчет при превышении порога памяти

ВЫВОД

В ходе выполнения лабораторной работы были изучены и применены на практике основные методы отладки Python-программ. Интерактивный отладчик `pdb` с его командами (`n` - next, `s` - step, `p` - print, `l` - list, `up/down` для навигации по стеку) оказался эффективным инструментом для пошагового выполнения кода и анализа состояния переменных в момент ошибки. Модуль `tracemalloc` позволил выявить утечки памяти, связанные с неограниченным ростом глобальных структур данных. Были исправлены четыре типа ошибок: добавлена проверка существования файла, реализовано ограничение размера кеша с использованием `lru_cache`, добавлена обработка исключений при преобразовании типов данных и при работе с файлами. Использование `breakpoint()` вместо `print()` для отладки оказалось более профессиональным подходом, так как позволяет интерактивно исследовать состояние программы без изменения ее логики. Полученные навыки могут быть применены при разработке и отладке любых Python-приложений, особенно при работе с файловыми операциями и большими объемами данных.